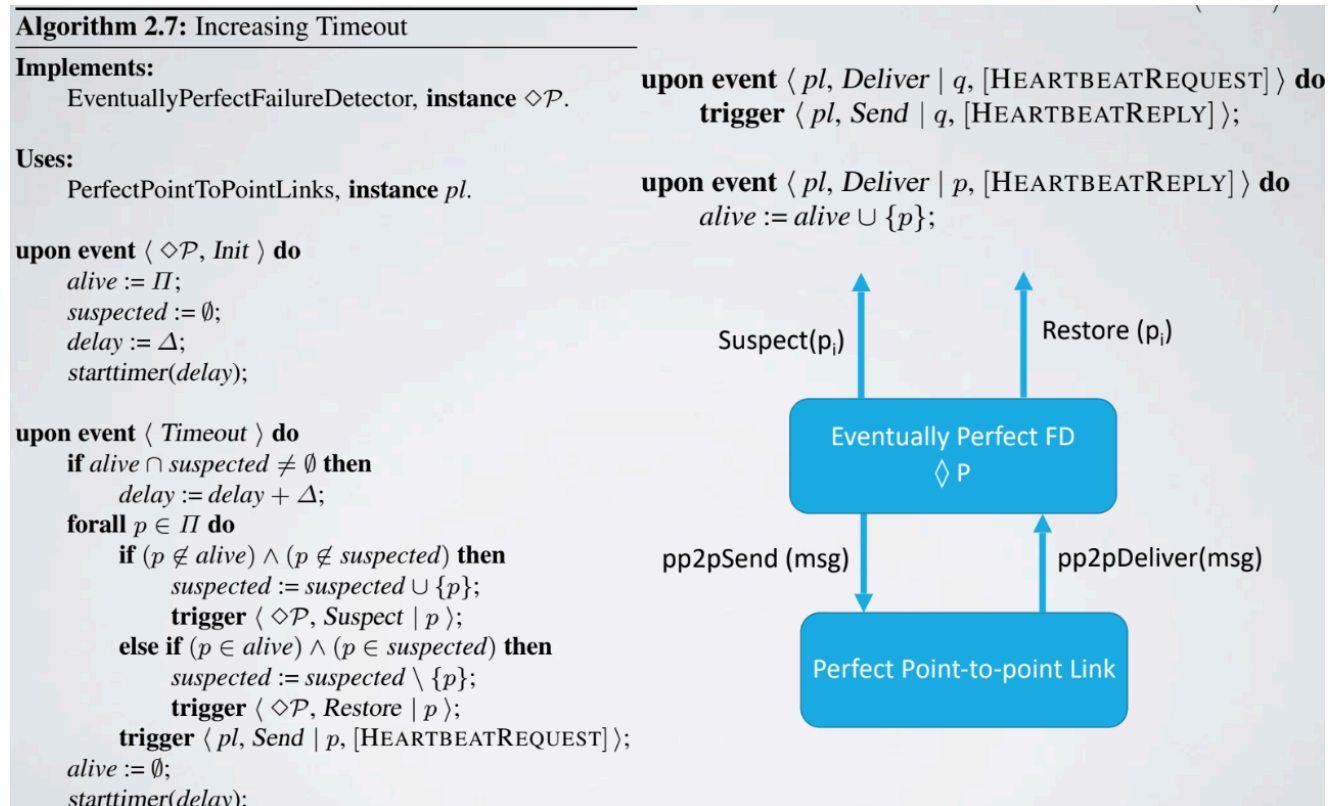


Distributed Systems - 13_10

For the next five Mondays, the lecture will be held in this room.

Now, let's see the code for the $\diamond P$.



This algorithm is quite similar to the one of P .

We have an alive and suspected set. In the suspected set we put processes that we suppose they're dead.

If a message coming from a process arrives delayed, then it won't be neither in the alive set or the suspected set, then we'll put the process in the suspected set.

When a new timeout occurs, if the message arrives delayed, we'll have that the process is both alive and suspected, thus we'll update the delay. Then, when we iterate in the for, we'll enter the if because the process is both alive and suspected, and remove the process from the suspected set, triggering the Restore event.

Let's say now that we have p_1, p_2, p_3 and p_2 at a certain point dies. Is it possible that p_1 will see a restore event for p_2 later after he died? Yes, the pong message sent by p_2 before he died could arrive with a delay. Though, the number of times I can restore it it's finite (the time the process p_2 has lived it's finite).

How do we modify the algorithm to avoid this kind of thing? We could use the concept of rounds, and by checking the round number of a message we can decide if that message is late.

Though, this is not necessarily a problem, because the property of strong completeness is implemented anyways.

Now let's see the proof, strong completeness.

If a process crashes, it will stop to send messages. Thus, the process will be suspected by every correct process and no process will revise the judgement.

Now let's see the second property.

Our systems works such that, after t , the system will always be synchronous. In the proof, we do not consider before time t , only after time t . After time t , a message sent by p to q will be delayed with a bound of MAX-DELAY. If p was suspected by q , then

q will revise his decision (the message has to arrive at some point if p is alive).

Moreover, q increase it's delay. By always increasing the value of Δ (because the process keeps being flagged as suspected and then corrected), there will be a point in which the value of delay will be $\geq 2 \cdot MAX$, so the message will have to arrive.

We've seen how the set of suspected processes will stabilize among processes.

LEMMA.

Let us suppose that we have S_1 and S_2 , with them being the set of suspected processes for p_1 and p_2 . Let's say that they are different. The only way to be different, there $\exists$ a process inside one but not the other, let's say $p \in S_1$ and $p \notin S_2$. If this process is a faulty process, then this situation violates strong completeness. So, the process can't be faulty because otherwise we would have $p \in S_2$. So p has to be correct process, but then we're violating the accuracy, because we're suspecting forever a process p , which should not be suspected because it's a correct process (at least, not forever). So S_1 and S_2 has to be the same.

The failure detectors are strong, they tell us correctly if a process has failed. This will help us building things.

We then have another primitive, with which everyone will agree that a process is correct, and this thing is known as Leader Election.

By having leader election, until the leader does not die, we can solve problems with a Client-Server pattern.

For the Leader Election, we have two algorithms. One is the Leader Election, the other the Eventual Leader Election. In *le*, we implement it for synchronous systems.

It has a single event $Leader(p_i)$, telling the upper layer that a process p_i is now the leader. If p_i dies, then the Leader event will be called again with another

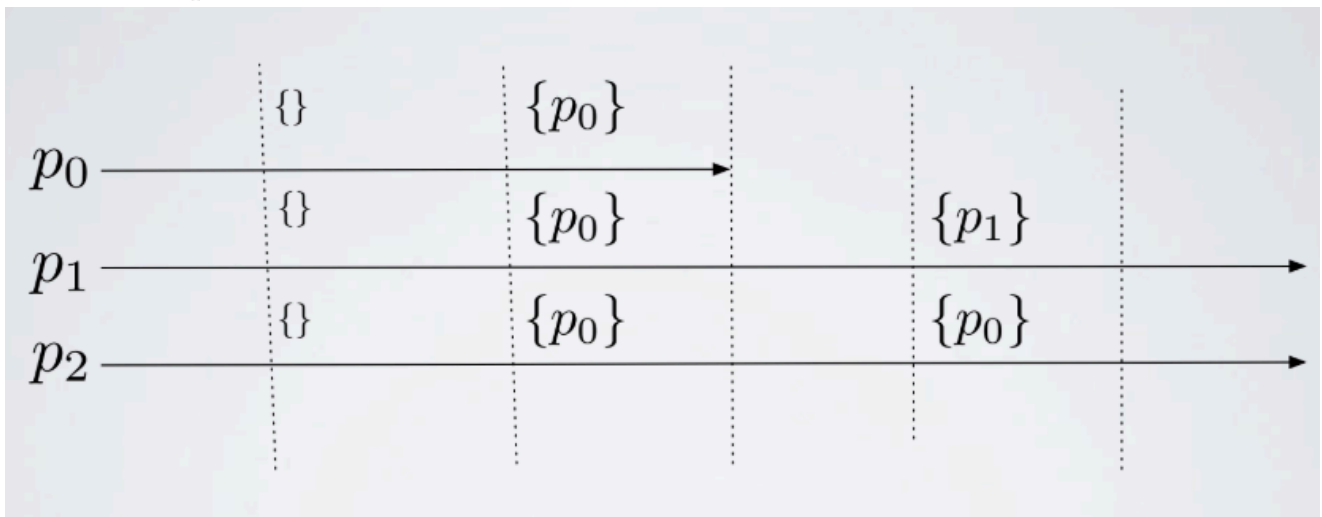
process.

It has two properties:

- (Eventual detection) Either there is no correct process, or some correct process is eventually elected as leader. *It helps us to have something that's not "lazy".*
- (Accuracy): If a process is leader, then all previously elected leaders have crashed. Let's say that we are p_1 , electing at the beginning p_4 , and then p_2 electing p_3 . If p_4 and p_3 is correct, this situation violates Accuracy, we are electing a new leader even if the other is still correct, so p_2 should elect p_4 as well. This also means that we can't change our mind electing a new leader, at least until it doesn't die.

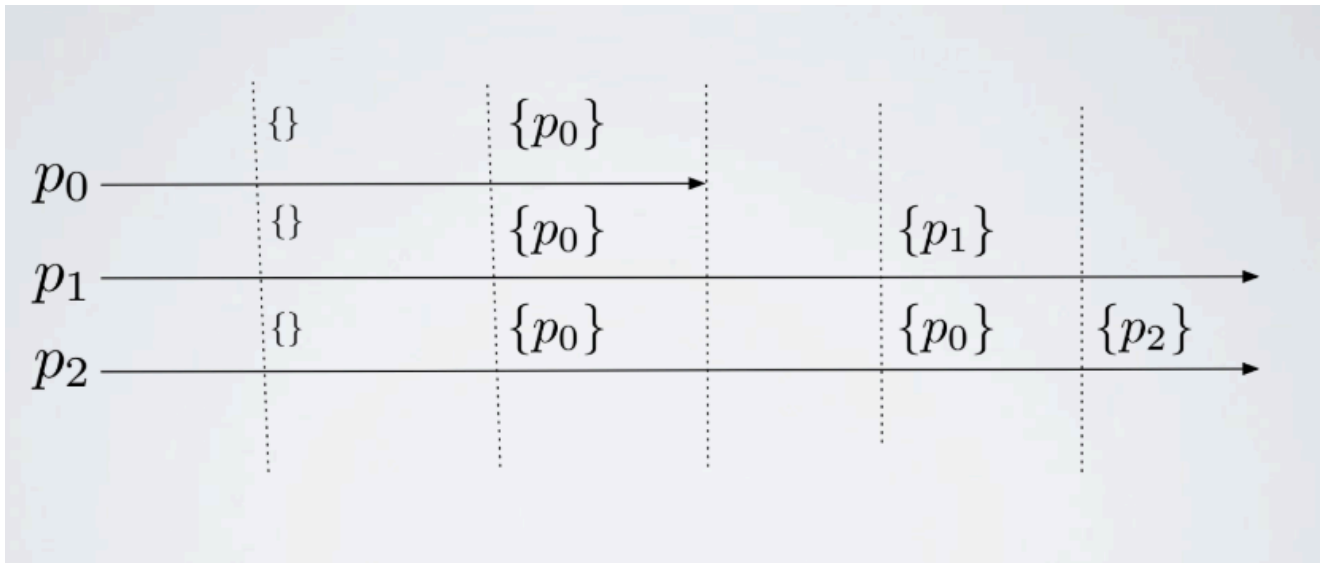
This property is "strange" because if two processes elect a leader at the exact same time, then "previously" doesn't work.

Is this case possible?



Yes, because p_2 is trusting a dead process, and we are not violating the property of eventual detection because only eventually we'll have to elect a new leader. If this is forever, however, this won't be correct.

Is this other case correct?



If p_2 elects a new leader, we should have that the new leader has to be the same one elected previously by another process. In the example we are violating the accuracy property.

How to implement this algorithm by using the perfect failure detector? We'll have a set of correct processes. If a process dies, we update the set, and elect the new leader by taking the process with the new minimum id.

Algorithm 2.6: Monarchical Leader Election

Implements:

LeaderElection, **instance** le .

Uses:

PerfectFailureDetector, **instance** $\mathcal{P}$.

upon event $\langle le, Init \rangle$ **do**

$suspected := \emptyset$;

$leader := \perp$;

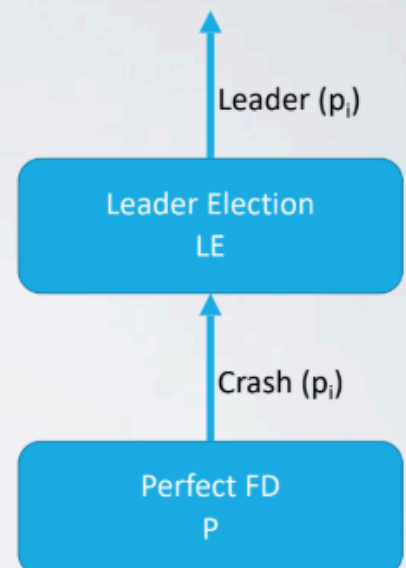
upon event $\langle \mathcal{P}, Crash \mid p \rangle$ **do**

$suspected := suspected \cup \{p\}$;

upon $leader \neq \maxrank(\Pi \setminus suspected)$ **do**

$leader := \maxrank(\Pi \setminus suspected)$;

trigger $\langle le, Leader \mid leader \rangle$;



Let's see the last handler. What is $\maxrank$ doing? Let's say that we have $\Pi = \{p_1, p_2, p_3\}$. Initially the leader is $\perp$. At the beginning we have $\perp \neq p_3$, then I

elect p_3 as a leader. If p_3 dies, we remove p_3 from Π , and take p_2 as the max rank, thus updating the leader.

What is the proof?

For the eventual detection, we have that it comes from the strong completeness of P .

Regarding the accuracy, we have the fact that we have the elect the same correct leader. So, this comes from the strong accuracy of P and the total order on ranks.

We could also take the minimum id rather than max, but everyone has to agree on that decision.

What happens with an eventual sync system (so we have a failure detector that is not perfect)? We use Ω . We'll have that, after a certain stabilization time, we'll have a "leader concordato".

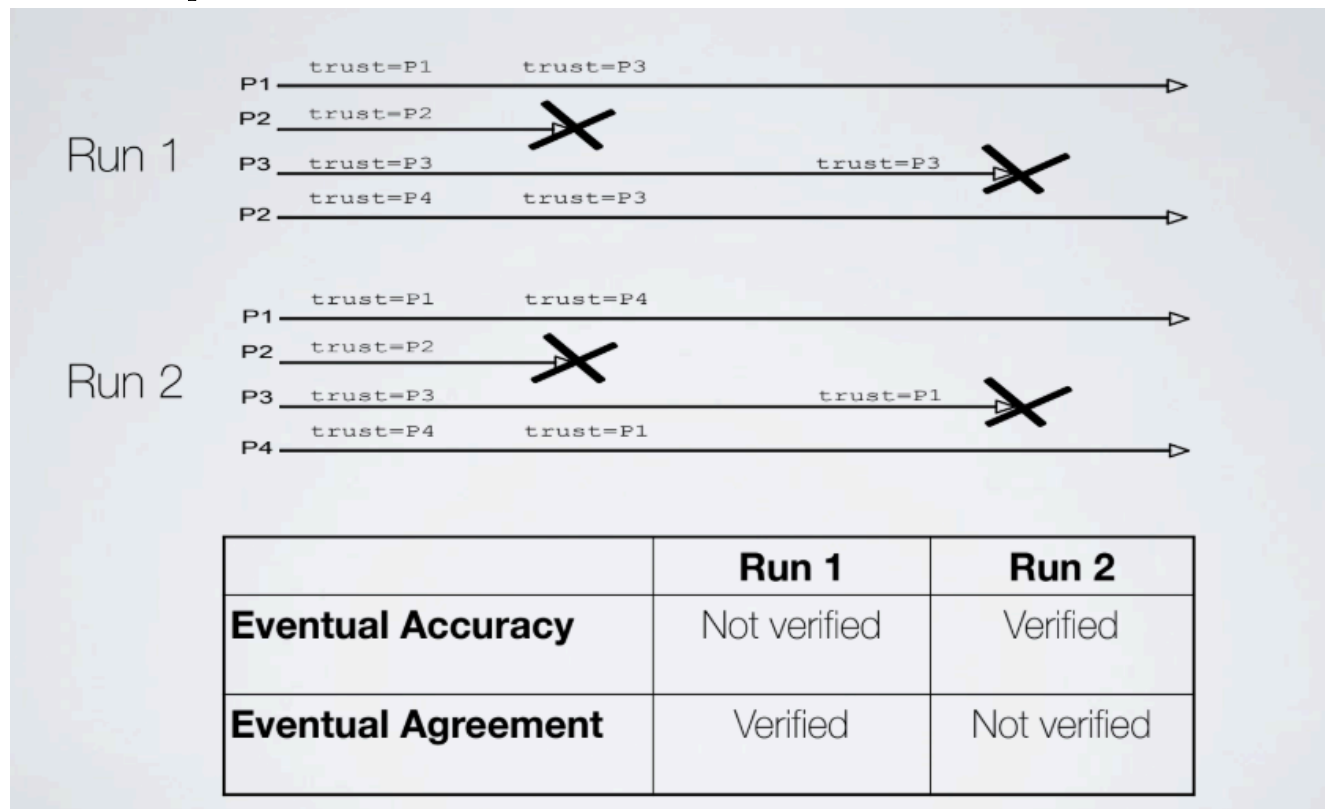
With Ω , we have a new event (only this one) named $\text{Trust}(p_i)$.

The properties:

- (Eventual accuracy): There is a time after which every correct process trusts some correct process. *There is a time in the future, when we can only trust someone who's in the correct set.*
- (Eventual agreement): There is a time after which no two correct processes trust different correct processes.

Can we modify the second property by removing correct? Yes, because by combining the first property and the second one modified, it would still work.

Let's see a possible execution:



Is run 1 correct? No, we are violating the eventual accuracy.

Run 2? Still not correct, we are violating the eventual agreement. We are trusting forever two different processes.

If the first run was finite, are we violating a property? No, because the properties are liveness.

How do we implement this? The code is similar to the previous one.

Algorithm 2.8: Monarchical Eventual Leader Detection

Implements:

EventualLeaderDetector, **instance** Ω .

Uses:

EventuallyPerfectFailureDetector, **instance** $\diamond P$.

upon event $\langle \Omega, \text{Init} \rangle$ **do**

$\text{suspected} := \emptyset;$

$\text{leader} := \perp;$

upon event $\langle \diamond P, \text{Suspect} \mid p \rangle$ **do**

$\text{suspected} := \text{suspected} \cup \{p\};$

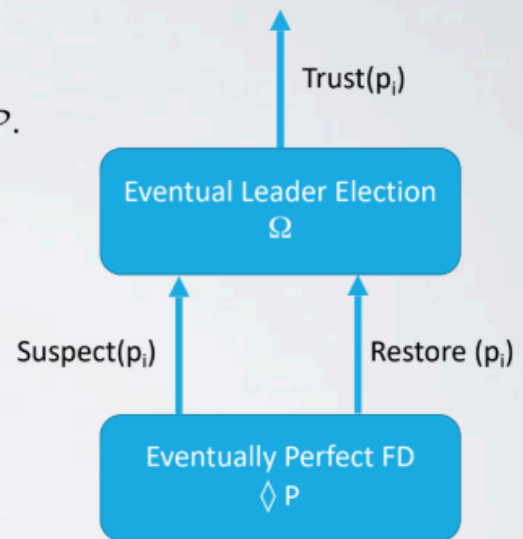
upon event $\langle \diamond P, \text{Restore} \mid p \rangle$ **do**

$\text{suspected} := \text{suspected} \setminus \{p\};$

upon $\text{leader} \neq \text{maxrank}(\Pi \setminus \text{suspected})$ **do**

$\text{leader} := \text{maxrank}(\Pi \setminus \text{suspected});$

trigger $\langle \Omega, \text{Trust} \mid \text{leader} \rangle;$



We just have the two handlers for $\diamond P$.

This algorithm is correct because the suspected set has a special property, that suspected sets in different processes will converge (lemma shown before).

The proofs:

ELD1 Eventual Accuracy: By the strong completeness of the FD we have that eventually suspected set contains all the crashed processes. Thus $\setminus p_i \setminus \text{suspected}$ contains only correct processes (or its empty).

ELD2 Eventual Agreement: For any pair of correct processes, their suspected sets eventually stabilises to the same content (by the property of the FD). If the set are equals $\setminus p_i \setminus \text{suspected}$ returns the same ID on both processes.

We take a look at three models now:

- Fail Silent: systems that cannot detect failures
- Fail Noisy: we have a perfect link with $\diamond P$
- Fail Stop: We have a perfect link with the usage of P

↓ With going down, the system is "stronger"

Also, if we are in a Fail Stop, we are also in a Fail Noisy, and then also in the Fail Silent. So, we have this kind of relationship: $\text{fail stop} \subset \text{fail noisy} \subset \text{fail silent}$. The one in which we can solve most problems is fail stop.

Let's say that we have a model that's fail stop, and one that is synchronous, what is the relationship between these two?

We have that $\text{fail stop} \subset \text{synch}$. In fact, clock synchronization can be solved in synchronous systems.

Now let's see some applications.

The first we see is the application of the Failure Detection to Lamport's Mutual Exclusion.

In order to do so, we need to modify the properties a bit.

Events:

- Request: From upper layer - Requests access to Critical Section (CS)
- Grant: To upper layer - Grant the access to CS.
- Release: From upper layer - Release the CS.

Properties:

- (Mutual Exclusions) At any time t , at most **one non-crashed process** is inside the CS.
- (Liveness) If a correct process p requests access, then it eventually enters the CS.
- (Fairness) If a correct process p requests access before a process q , then q cannot access the CS before p .

We need to modify the mutual exclusion property (as well as others), by putting that the process in the CS should not be crashed, otherwise if a process enters and dies inside, then it won't send a Release and deadlocks the critical section.

In liveness, I should add correct, because a process might ask to enter and then crash, and so it's not possible that he enters.

Now, given the original algorithm, we want to modify it by making it fault tolerant. We have to identify the critical points in which, if a failure happens, the system breaks.

Like, if someone is in the critical section and fails, there will be a deadlock. So one critical point is the release function.

Another point is the condition in the handler at line 19, the one of the ACKs.

To fix that, we could only wait for acks coming from processes that are not crashed yet. To do so, we'll create an handler for crash of a process. And in the

condition, we'll have $\forall p \in \Pi \setminus \text{crashed}$.

For the other problem, if a process has crashed I check at the crashed set, and if he has a request I remove it.

Let's take a look at the patch.

Algorithm 15 Lamport's ME Algorithm on process p_i - Fault-Tolerant Patch

```
1: upon event INIT
2:    $Requests = Acks = \emptyset$ 
3:    $scalar\_clock = 0$ 
4:    $my\_req = \perp$ 
5:    $\Pi = \{p_0, p_1, \dots, p_{n-1}\}$  ▷ Set of all processes
6:    $Crashed = \{\}$ 

7: ▷ Crash handler, the crash event is generated by a perfect failure detector P
8: upon event CRASH(process  $p_j$ )
9:    $Crashed = Crashed \cup \{p_j\}$ 
10:  for all  $req \in Requests$  such that  $p_j$  is the sender of  $req$  do
11:    remove  $req$  from  $Requests$ 

12: ▷  $ts(x) < ts(y)$  when  $scalar\_clock$  of  $x$  is less than the one of  $y$ , or they are equal and the id that sent  $x$  is less than the id that sent  $y$ 
13: upon event  $\nexists req \in Requests : ts(req) < ts(my\_req) \wedge \forall p \in (\Pi \setminus Crashed) : \exists m \in Acks | ts(m) > ts(my\_req)$ 
14:  trigger event GRANTED
```

If we receive from P a crash event, we update the crashed set, and remove the requests coming from him.

But is it enough? No, what happens if the request arrives after we detected that the process has crashed? We'll handle the request and process it, even if the process is dead, leading to a deadlock.

To fix this, I could simply discard a request coming from a crashed process, in line 23.

So we have this new patch:

Algorithm 15 Lamport's ME Algorithm on process p_i - Fault-Tolerant Patch

```

1: upon event INIT
2:    $Requests = Acks = \emptyset$ 
3:    $scalar\_clock = 0$ 
4:    $my\_req = \perp$ 
5:    $\Pi = \{p_0, p_1, \dots, p_{n-1}\}$  ▷ Set of all processes
6:    $Crashed = \{\}$ 

7: ▷ Crash handler, the crash event is generated by a perfect failure detector P
8: upon event CRASH(process  $p_j$ )
9:    $Crashed = Crashed \cup \{p_j\}$ 
10:  for all  $req \in Requests$  such that  $p_j$  is the sender of  $req$  do
11:    remove  $req$  from  $Requests$ 

12: ▷  $ts(x) < ts(y)$  when  $scalar\_clock$  of  $x$  is less than the one of  $y$ , or they are equal and the id that sent  $x$  is less than the id that sent  $y$ 
13: upon event  $\nexists req \in Requests : ts(req) < ts(my\_req) \wedge \forall p \in (\Pi \setminus Crashed) : \exists m \in Acks | ts(m) > ts(my\_req)$ 
14:   trigger event GRANTED

15: upon event DELIVER MESSAGE( $m$ )
16:   if  $m$  is a REQ and the sender of  $m$  is not in  $Crashed$  then
17:      $scalar\_clock = \max(clock(m), scalar\_clock) + 1$ 
18:      $Request\_set = Request\_set \cup \{m\}$ 
19:      $scalar\_clock = scalar\_clock + 1$ 
20:     SEND FIFOPERFECTLINK( $sender(m)$ , ( $ACK, ts = < i, scalar\_clock >$ ))
21:   else if  $m$  is a ACK then
22:     ...

```

Let's now look at another algorithm. Always a mutual exclusion algorithm, but this time is built on p2p, P and le . Because of the fact that we have leader election, we could build it so that the leader grants access to the CS.

Let's say that we have p_0 as a leader, and also p_1, p_2, p_3 . p_0 takes track of all incoming requests. Let's say that he receives at first a request from p_1 , then from p_2 . p_1 is first in the queue, then he sends a grant to p_1 . When p_1 is done, he sends a release to p_0 , and p_0 removes p_1 from the queue, proceeding with p_2 .

when at the exam we have a system with which we can use a leader, use the leader as much as possible. also, by having P , we can build le

What happens now if one of the processes (that are not the leader) fails? Let's say that p_2 crashes when he is inside the CS. *i didn't get this part much*

What happens if the leader crashes? We have to elect a new leader, which should be automatic with le . We have to know, as a new leader, however, if someone was in the critical section. So we put the requests (even the redirected ones) in a wait until he has a clear view of the system. To do so, he sends a broadcast to everyone asking if they are in the critical section, waiting for an answer only from processes in $\Pi/crashed$.

Starting from the description, create an algorithm.

Describe that can be solved in sync but not in fail-stop:

- clock sync

Describe one problem that can be solved in fail-stop but not in sync:

- $\emptyset$